

Universidad Nacional del Comahue
Centro Regional Zona Atlántica
Tecnicatura Superior en Administración de Sistemas y Software Libre

Automatización y Scripting

Trabajo Práctico N.º 2

Scripts avanzados en Bash

Alumno:	Rolando Cobis
Legajo:	CURZA-9389
DNI:	96.495.580
Token de Autenticidad:	93895580
Repositorio:	<code>github.com/cRolandoJr/tp2-cobis-9389</code>
Docente:	Ramiro García Poggi
Fecha de entrega:	17 de septiembre de 2026

Índice

1. Enunciado del Trabajo Práctico	2
1.1. Tarea 1: Buscador de versiones en PDF (pdf_version.sh)	2
1.2. Tarea 2: Renombrador y organizador automático (organizador.sh)	2
1.3. Tarea 3: Monitoreo interactivo de recursos (monitorear.sh)	2
1.4. Tarea 4: Verificador de sitios web (chequear_paginas.sh)	3
1.5. Tarea 5: Respalgador con prevención de ejecución simultánea (backup_manager.sh)	3
1.6. Consideraciones acerca de los scripts	3
2. Hipótesis, supuestos y aclaraciones	4
2.1. Entorno de ejecución	4
2.2. La ejecución del respaldo requiere privilegios	4
2.3. Un fallo de mkdir no implica que el bloqueo esté tomado	4
2.4. Alcance de «tiempo real» en el monitoreo de RAM	4
2.5. Criterio del filtro de iniciales	4
2.6. Otras decisiones menores	5
3. Resolución	6
3.1. Tarea 1: Buscador de versiones en PDF	6
3.2. Tarea 2: Renombrador y organizador automático	7
3.3. Tarea 3: Monitoreo interactivo de recursos	9
3.4. Tarea 4: Verificador de sitios web	11
3.5. Tarea 5: Respalgador con prevención de ejecución simultánea	13
4. Pruebas realizadas	15

1. Enunciado del Trabajo Práctico

Objetivo. Implementar scripts en Bash de nivel intermedio-avanzado, utilizando bucles complejos, arreglos, manipulación de cadenas en variables, control de procesos con archivos de bloqueo (lockfiles), interacción con servicios web externos mediante cURL, y creación de interfaces interactivas en terminal (select y case).

Descripción del caso práctico. Como administrador de la red de laboratorios del CURZAS, se le solicita automatizar la gestión y monitoreo diario de los servidores de archivos de la universidad. Debe estructurar scripts que procesen extensiones de archivos de los alumnos, verifiquen el estado de servicios web locales, limpien directorios temporales de forma ordenada y prevengan colisiones al ejecutar tareas de respaldo redundantes.

1.1. Tarea 1: Buscador de versiones en PDF (pdf_version.sh)

Los archivos PDF almacenan su versión de formato en la primera línea de texto plano (ej: %PDF-1.4).

- Escriba un script que busque todos los archivos con extensión .pdf en el directorio actual y subdirectorios.
- Por cada archivo encontrado, extraiga la primera línea utilizando el comando head -n 1.
- Imprima el nombre del archivo y su versión en formato: Archivo: [nombre.pdf] - Versión PDF: [1.X].
- Personalización: el script debe filtrar y omitir cualquier archivo PDF cuyo nombre contenga la palabra «excluir» o las iniciales del alumno.

1.2. Tarea 2: Renombrador y organizador automático (organizador.sh)

Cree un script que ordene y clasifique archivos en carpetas según su extensión.

1. Requisitos: debe recibir un directorio destino como argumento. Validar que el directorio exista y sea accesible. Si no se pasa o no existe, fallar con código 1.
2. Lógica:
 - Crear dentro del directorio destino las carpetas imagenes/ (para .jpg, .png), documentos/ (para .pdf, .txt, .docx) y comprimidos/ (para .zip, .tar.gz, .rar).
 - Mover cada archivo a la carpeta correspondiente utilizando bucles for y condicionales.
 - Si encuentra archivos con extensión .old, renombrarlos masivamente eliminando la extensión .old y reemplazándola por .backup usando la sustitución de cadenas en variables de Bash (\${archivo%.old}.backup).
 - Los archivos que no encajen en ninguna categoría deben moverse a otros/.

1.3. Tarea 3: Monitoreo interactivo de recursos (monitorear.sh)

Implemente un panel interactivo utilizando select y case.

1. El script debe mostrar un menú interactivo con las siguientes opciones:
 - Opción 1, monitorear memoria RAM: muestra la RAM libre y usada en megabytes en tiempo real.
 - Opción 2, buscar archivos grandes: busca archivos mayores a 10 MB en el directorio \$HOME y muestra los 5 más grandes.
 - Opción 3, espacio en particiones: muestra el uso de disco formateado de las particiones físicas montadas.
 - Opción 4, salir: termina la ejecución con un saludo personalizado con el legajo del alumno.
2. El menú debe repetirse indefinidamente hasta que el usuario elija la opción de salir.

1.4. Tarea 4: Verificador de sitios web (chequear_paginas.sh)

Cree un script que verifique la disponibilidad de servidores mediante HTTP.

1. Entrada: el script debe aceptar una lista de URL separadas por espacios como argumentos. Si no se ingresa ninguna, debe leer las URL desde un archivo de texto llamado `sitios_[LEGAJO].txt` (que debe crear previamente en el repositorio).
2. Lógica: para cada URL, realice una petición HTTP silenciosa usando `cURL` para obtener únicamente el código de estado. Imprima la URL y su código en colores en la terminal: verde si el código es 200, amarillo si es un redireccionamiento 3xx, rojo si es 4xx o 5xx.
3. Guardar el reporte en `logs/chequeo_[LEGAJO].log`.

1.5. Tarea 5: Respalador con prevención de ejecución simultánea (backup_manager.sh)

Cree un script de respaldo seguro para evitar que dos instancias del script se ejecuten al mismo tiempo (lo cual corrompería el backup).

1. Mecanismo de lockfile:
 - Utilice la creación atómica de directorios (`mkdir "/var/lock/backup_[LEGAJO].lock"`) al inicio del script para actuar como semáforo de bloqueo.
 - Si el comando `mkdir` falla, significa que el script ya se está ejecutando. Debe imprimir un mensaje de error y terminar inmediatamente con código 9.
 - Si tiene éxito, asegure la eliminación del directorio de bloqueo en caso de finalización normal o interrupción imprevista utilizando `trap`.
2. Respaldo:
 - El script debe buscar todos los archivos en su directorio de código modificados en las últimas 24 horas (`find . -mtime -1 -type f`).
 - Copiar los archivos encontrados a un directorio temporal llamado `/tmp/backup_[LEGAJO]/`.
 - Empaquetar y comprimir dicho directorio en un archivo `tarball` llamado `backup_[LEGAJO]_[FECHA].tar.gz` dentro de la carpeta `logs/`.

1.6. Consideraciones acerca de los scripts

- La interfaz solo podrá utilizar modo texto plano.
- Salvo indicación expresa no puede utilizarse código externo de ningún tipo.
- El script no deberá abortar en ningún momento de su ejecución. Por este motivo se exige la validación de todos los datos de entrada.
- Debe haber un historial de al menos 3 commits con mensajes claros.

2. Hipótesis, supuestos y aclaraciones

2.1. Entorno de ejecución

Los cinco scripts fueron desarrollados y probados sobre **NixOS** con **Bash 5.3.15**, no sobre el Ubuntu que asume la cátedra. De ahí se desprenden dos desvíos, ambos deliberados:

- **El shebang es `#!/usr/bin/env bash` y no `#!/bin/bash`.** En NixOS los intérpretes viven bajo `/nix/store` y `/bin/bash` no existe, de modo que la ruta absoluta produciría un error de intérprete no encontrado. La forma con `env` resuelve el binario por `PATH` y funciona en las dos distribuciones.
- **El directorio `/var/lock` pertenece a `root`.** Se detalla en el apartado siguiente.

2.2. La ejecución del respaldo requiere privilegios

El enunciado fija la ruta del bloqueo en `/var/lock/backup_9389.lock`. En este sistema `/var/lock` es un enlace simbólico a `/run/lock`, propiedad de `root` y con permisos `drwxr-xr-x`, por lo que un usuario común no puede crear nada allí:

```
$ ls -ld /var/lock
lrwxrwxrwx - root 16 may 09:12 /var/lock -> ../run/lock

$ mkdir /var/lock/_probe_9389
mkdir: cannot create directory '/var/lock/_probe_9389': Permiso denegado
```

Se conservó la ruta que indica el enunciado y el script se ejecuta con `sudo`. Eso trae una consecuencia que hubo que atender: todo lo que el script escribe queda a nombre de `root`, incluido el tarball que cae dentro del repositorio, y el usuario no podría borrarlo ni versionarlo. Por eso el script devuelve la propiedad de los artefactos a `$SUDO_USER` antes de terminar.

2.3. Un fallo de `mkdir` no implica que el bloqueo esté tomado

El enunciado dice que si `mkdir` falla es porque el script ya se está ejecutando. Eso es cierto solo en el caso normal: `mkdir` falla también por falta de permisos sobre `/var/lock`, que es exactamente lo que ocurre al olvidar el `sudo`. Informar que hay una instancia en ejecución en ese caso sería un diagnóstico falso, con un código de salida que lo respalda.

El script distingue los dos casos preguntando, después del fallo, si el directorio de bloqueo existe. Si existe, el bloqueo está tomado y sale con **9**, como pide el enunciado. Si no existe, el problema es de permisos y sale con **7**, informando que hace falta `sudo`.

2.4. Alcance de «tiempo real» en el monitoreo de RAM

La opción 1 del panel toma cinco lecturas separadas por dos segundos en lugar de refrescar de forma continua. Un `free -m -s 2` no termina nunca por sí solo: exige un `Ctrl-C` del usuario, y el menú, que debe repetirse hasta que se elija salir, no volvería a mostrarse. El muestreo acotado deja ver la variación de la memoria y devuelve el control al menú.

2.5. Criterio del filtro de iniciales

El filtro de la Tarea 1 compara la cadena exacta `RC`, sensible a mayúsculas. Un filtro insensible sobre `rc` omitiría archivos legítimos como `marco.pdf` o `fuentes.pdf`, porque el patrón también coincide dentro de una palabra. En la captura de la Tarea 1 puede verse que `marco.pdf` sí se analiza y `informe-RC.pdf` no.

2.6. Otras decisiones menores

- **El case de la Tarea 2 evalúa * . tar . gz antes que cualquier patrón que también coincida con . gz**, porque corta en la primera coincidencia y una doble extensión se clasificaría mal en el orden inverso.
- **La Tarea 4 no sigue las redirecciones** (no se usa `curl -L`). Siguiéndolas se vería el 200 del destino final y el 3xx nunca aparecería, que es justamente uno de los tres colores pedidos.
- **Se agregó un cuarto estado a la Tarea 4 para el código 000**, que cURL devuelve cuando no hubo respuesta alguna (el DNS no resuelve, la conexión se rechaza o expira el tiempo de espera). No es un código HTTP, así que no corresponde tratarlo como error del servidor.
- **Las etiquetas de estado de la Tarea 4 se escriben sin tildes**. `printf` rellena las columnas contando bytes y en UTF-8 una vocal acentuada ocupa dos, de modo que la palabra correspondiente al 3xx corría la columna siguiente y desalineaba esa única fila.
- **Los recorridos de archivos usan `find -print0` con `read -d ''`**. Recorrer la salida de `find` con un `for` común parte en pedazos los nombres que contienen espacios.
- **El archivo `sitios_9389.txt` contiene dominios de la Universidad Nacional del Comahue** elegidos para cubrir los cuatro estados que distingue el script.

3. Resolución

3.1. Tarea 1: Buscador de versiones en PDF

El script recibe opcionalmente un directorio (por omisión, el actual), valida que exista y sea accesible, y recorre el árbol buscando archivos .pdf. De cada uno lee la primera línea con `head -n 1` y recorta la versión con una expresión regular, porque el resto del archivo es binario y no se puede confiar en el corte de línea. Los archivos filtrados y los que no declaran una cabecera válida se informan sin interrumpir el recorrido.

3.1.1. Código de pdf_version.sh

```
#!/usr/bin/env bash
# pdf_version.sh - Tarea 1: informa la versión de formato de cada PDF del árbol.
# Uso      : ./pdf_version.sh [directorio]    (por defecto, el directorio actual)
# Códigos: 0 si anduvo
#          1 si el directorio que le paso no existe o no puedo entrar

DIRECTORIO_BASE="${1:-.}"
INICIALES="RC"
PALABRA_EXCLUIR="excluir"

if [ ! -d "${DIRECTORIO_BASE}" ] || [ ! -x "${DIRECTORIO_BASE}" ]; then
    echo "ERROR: '${DIRECTORIO_BASE}' no es un directorio accesible." >&2
    exit 1
fi

encontrados=0
omitidos=0

while IFS= read -r -d '' archivo; do          # -print0 y -d '' por los nombres con espacios
    nombre="$(basename "${archivo}")"

    # "RC" exacto y sensible a mayúsculas: con "rc" caerían marco.pdf y fuentes.pdf
    if [[ "${nombre}" == "${INICIALES}"* || "${nombre}" == "${PALABRA_EXCLUIR}"* ]]; then
        omitidos=$(( omitidos + 1 ))
        continue
    fi

    if [ ! -r "${archivo}" ]; then
        echo "AVISO: '${nombre}' no se puede leer, se omite." >&2
        continue
    fi

    # El resto del archivo es binario: recorto con regex en vez de confiar en el corte de línea
    primera_linea="$(head -n 1 "${archivo}" 2>/dev/null)"

    if [[ "${primera_linea}" =~ %PDF-([0-9]+\.[0-9]+) ]]; then
        version="${BASH_REMATCH[1]}"
        echo "Archivo: [${nombre}] - Versión PDF: [${version}]"
        encontrados=$(( encontrados + 1 ))
    else
        echo "AVISO: '${nombre}' no declara una cabecera %PDF válida." >&2
    fi
done < <(find "${DIRECTORIO_BASE}" -type f -name '*.pdf' -print0 2>/dev/null)

echo
echo "Total analizados: ${encontrados} | Omitidos por filtro: ${omitidos}"
exit 0
```

3.1.2. Salida obtenida

Sobre un árbol de prueba con cuatro PDF, dos de ellos alcanzados por el filtro:

```

.../tp2-cobis-9389  main ?t 13:03
→ mkdir -p /tmp/demo_pdf/sub
.../tp2-cobis-9389  main ?t 13:23
→ cp ~/Clases/Desarrollo_Web_1erAño_2doCua/Concep_bd/Apunte_Unidad_1.pdf /tmp/demo_pdf/apunte.pdf
.../tp2-cobis-9389  main ?t 13:24
→
cp
.../tp2-cobis-9389  main ?t 13:24
→ cp /tmp/demo_pdf/apunte.pdf /tmp/demo_pdf/marco.pdf
.../tp2-cobis-9389  main ?t 13:26
→ cp /tmp/demo_pdf/apunte.pdf /tmp/demo_pdf/sub/informe-RC.pdf
.../tp2-cobis-9389  main ?t 13:27
→ cp /tmp/demo_pdf/apunte.pdf /tmp/demo_pdf/sub/algo-excluir.pdf
.../tp2-cobis-9389  main ?t 13:28
→ ./codigo/pdf_version.sh /tmp/demo_pdf
Archivo: [apunte.pdf] - Versión PDF: [1.3]
Archivo: [marco.pdf] - Versión PDF: [1.3]
Total analizados: 2 | Omitidos por filtro: 2

```

Figura 1: Ejecución de pdf_version.sh. marco.pdf se analiza; informe-RC.pdf y algo-excluir.pdf se omiten.

3.2. Tarea 2: Renombrador y organizador automático

El script valida que se haya pasado exactamente un argumento, que sea un directorio existente y que se pueda escribir dentro de él; en cualquier otro caso termina con código 1. Luego crea las cuatro carpetas de destino, renombra los archivos .old con la sustitución \${archivo%.old}.backup y recorre el directorio clasificando por extensión.

El renombrado va antes de la clasificación por una razón práctica: después de mover los archivos habría que ir a buscarlos a las cuatro carpetas nuevas.

3.2.1. Código de organizador.sh

```

#!/usr/bin/env bash
# organizador.sh - Tarea 2: clasifica archivos en carpetas según su extensión.
# Uso      : ./organizador.sh <directorio_destino>
# Códigos: 0 si anduvo
#          1 si no le paso el directorio, o no existe, o no puedo escribir en él

if [ "$#" -ne 1 ]; then
    echo "ERROR: se esperaba exactamente 1 argumento y se recibieron $#." >&2
    echo "Uso: $0 <directorio_destino>" >&2
    exit 1
fi

DESTINO="$1"

if [ ! -d "${DESTINO}" ]; then
    echo "ERROR: '${DESTINO}' no existe o no es un directorio." >&2
    exit 1
fi

if [ ! -w "${DESTINO}" ] || [ ! -x "${DESTINO}" ]; then
    echo "ERROR: no tengo permiso para escribir dentro de '${DESTINO}'." >&2
    exit 1
fi

for carpeta in imagenes documentos comprimidos otros; do
    if ! mkdir -p "${DESTINO}/${carpeta}"; then
        echo "ERROR: no pude crear '${DESTINO}/${carpeta}'." >&2
        exit 1
    fi
done

renombrados=0

```

```

movidos=0

# Los .old van primero: después de mover habría que buscarlos en las cuatro carpetas
for archivo in "${DESTINO}"/*.old; do
    [ -e "${archivo}" ] || continue          # el glob sin coincidencias queda literal
    nuevo="${archivo%.old}.backup"
    if mv -n -- "${archivo}" "${nuevo}"; then
        echo "Renombrado: ${basename "${archivo}"} -> ${basename "${nuevo}"}"
        renombrados=$(( renombrados + 1 ))
    else
        echo "AVISO: no pude renombrar '${basename "${archivo}"}'.'" >&2
    fi
done

for archivo in "${DESTINO}"/*; do
    [ -f "${archivo}" ] || continue          # las carpetas recién creadas no se tocan

    nombre="$(basename "${archivo}")"

    # *.tar.gz va antes que cualquier patrón que también matchee .gz: el case corta en la primera
    case "${nombre}" in
        *.jpg|*.jpeg|*.png)      carpeta="imagenes" ;;
        *.pdf|*.txt|*.docx)      carpeta="documentos" ;;
        *.tar.gz|*.zip|*.rar)    carpeta="comprimidos" ;;
        *)                       carpeta="otros" ;;
    esac

    if mv -n -- "${archivo}" "${DESTINO}/${carpeta}/"; then
        echo "Movido: ${nombre} -> ${carpeta}/"
        movidos=$(( movidos + 1 ))
    else
        echo "AVISO: no pude mover '${nombre}' (¿ya existe en destino?)." >&2
    fi
done

echo
echo "Renombrados: ${renombrados} | Movidos: ${movidos}"
exit 0

```

3.2.2. Salida obtenida

```

~/tp2-cobis-9389 main ? 14:44
→ mkdir -p /tmp/demo_org
~/tp2-cobis-9389 main ? 14:44
→ cd /tmp/demo_org
/tmp/demo_org 14:44
→ touch foto.JPG grafico.png informe.pdf notas.txt cata.docx paquete.tar.gz respaldo.zip viejo.rar datos.csv config.o
ld
/tmp/demo_org 14:45
→ cd ~/Clases/Admin_Sist_SL_2doAño_2doCua/AutomaScript/tp2-cobis-9389

~/tp2-cobis-9389 main ? 14:46
→ ./codigo/organizador.sh /tmp/demo_org
Renombrado: config.old -> config.backup
Movido: cata.docx -> documentos/
Movido: config.backup -> otros/
Movido: datos.csv -> otros/
Movido: foto.JPG -> imagenes/
Movido: grafico.png -> imagenes/
Movido: informe.pdf -> documentos/
Movido: notas.txt -> documentos/
Movido: paquete.tar.gz -> comprimidos/
Movido: respaldo.zip -> comprimidos/
Movido: viejo.rar -> comprimidos/

Renombrados: 1 | Movidos: 10

```

Figura 2: Ejecución de organizador .sh sobre diez archivos de prueba y un .old.

3.3. Tarea 3: Monitoreo interactivo de recursos

El panel se construye con `select`, que se encarga de imprimir el menú y repetirlo hasta que se ejecute un `break`. El `case` discrimina sobre el texto que devuelve `select` y no sobre el número tecleado, de modo que un cambio en el orden del menú no rompe las ramas.

Dos situaciones requieren atención porque el script no debe abortar: un número fuera de rango, que deja la variable vacía y se informa como opción inválida, y un `Ctrl-D`, que deja `REPLY` vacío y haría que el menú se repitiera indefinidamente sin forma de cerrarlo.

3.3.1. Código de `monitorear.sh`

```
#!/usr/bin/env bash
# monitorear.sh - Tarea 3: panel interactivo de monitoreo de recursos.
# Uso      : ./monitorear.sh
# Códigos: 0 siempre; sale por la opción "Salir" o por EOF.

LEGAJO="9389"
ALUMNO="Rolando Cobis"
MUESTRAS_RAM=5
INTERVALO_RAM=2

PS3=$'\n'"Elija una opción (1-4): "
```

```
# Muestreo acotado en vez de free -s: con un bucle infinito el menú no vuelve nunca
monitorear_ram() {
    echo "Memoria RAM: ${MUESTRAS_RAM} lecturas cada ${INTERVALO_RAM}s (en MB)"
    printf '%-10s %-10s %-10s %-10s\n' "Lectura" "Total" "Usada" "Libre"

    local i total usada libre
    for (( i = 1; i <= MUESTRAS_RAM; i++ )); do
        # Las tres columnas de una sola invocación: tres llamadas darían tres fotos distintas
        read -r total usada libre <<(free -m | awk '/^Mem:/ {print $2, $3, $4}')
        printf '%-10s %-10s %-10s %-10s\n' "${i}/${MUESTRAS_RAM}" "${total}" "${usada}" "${libre}"
        [ "${i}" -lt "${MUESTRAS_RAM}" ] && sleep "${INTERVALO_RAM}"
    done
}

buscar_archivos_grandes() {
    echo "Los 5 archivos de más de 10 MB más grandes dentro de ${HOME}:"

    # %s en bytes para poder ordenar numéricamente; numfmt lo vuelve legible recién al final
    local resultado
    resultado="$(find "${HOME}" -type f -size +10M -printf '%s\t%p\n' 2>/dev/null \
        | sort -rn | head -n 5)"

    if [ -z "${resultado}" ]; then
        echo " (no se encontraron archivos de más de 10 MB)"
        return
    fi

    printf '%s\n' "${resultado}" | while IFS=$'\t' read -r bytes ruta; do
        printf ' %8s %s\n' "${numfmt --to=iec --suffix=B "${bytes}"}" "${ruta}"
    done
}

espacio_particiones() {
    echo "Uso de disco de las particiones físicas montadas:"
    # El filtro por /dev/ deja afuera tmpfs y los montajes del store, que no son particiones
    df -h -x tmpfs -x devtmpfs | awk 'NR == 1 || /^\/dev\/\/'
}

echo "=== Panel de monitoreo - ${ALUMNO} (legajo ${LEGAJO}) ==="

select opcion in "Monitorear memoria RAM" \
    "Buscar archivos grandes" \
    "Espacio en particiones" \
    "Salir"; do
    # Un Ctrl-D deja REPLY vacío y select repetiría el menú para siempre
    if [ -z "${REPLY}" ]; then
```

```
    echo
    echo "Entrada cerrada. Hasta luego, ${ALUMNO} (legajo ${LEGAJO})."
    break
fi

# El case va sobre el texto y no sobre el número: sobrevive a un cambio de orden del menú
case "${opcion}" in
    "Monitorear memoria RAM")  echo; monitorear_ram      ;;
    "Buscar archivos grandes") echo; buscar_archivos_grandes ;;
    "Espacio en particiones")  echo; espacio_particiones   ;;
    "Salir")
        echo
        echo "Hasta luego, ${ALUMNO} (legajo ${LEGAJO}). Panel cerrado."
        break
        ;;
    *)
        echo "Opción inválida: '${REPLY}'. Elija un número del 1 al 4." >&2
        ;;
esac
done

exit 0
```

3.3.2. Salida obtenida

```

.../tp2-cobis-9389  main ?  14:47
→ ./codigo/monitorear.sh
== Panel de monitoreo - Rolando Cobis (legajo 9389) ==
1) Monitorear memoria RAM
2) Buscar archivos grandes
3) Espacio en particiones
4) Salir

Elija una opción (1-4): 1

Memoria RAM: 5 lecturas cada 2s (en MB)
Lectura      Total      Usada      Libre
1/5           31287      7868      13483
2/5           31287      7869      13483
3/5           31287      7869      13483
4/5           31287      7869      13483
5/5           31287      7869      13483

Elija una opción (1-4): 3

Uso de disco de las particiones físicas montadas:
S.ficheros      Tamaño Usados  Disp  Uso%  Montado en
/dev/mapper/cryptroot  931G  666G  259G  73%  /
/dev/mapper/cryptroot  931G  666G  259G  73%  /var/log
/dev/mapper/cryptroot  931G  666G  259G  73%  /nix
/dev/mapper/cryptroot  931G  666G  259G  73%  /btrfs
/dev/mapper/cryptroot  931G  666G  259G  73%  /.snapshots
/dev/mapper/cryptroot  931G  666G  259G  73%  /home
/dev/nvme0n1p1        1022M  84M   939M   9%  /boot

Elija una opción (1-4): 9
Opción inválida: '9'. Elija un número del 1 al 4.

Elija una opción (1-4): 4

Hasta luego, Rolando Cobis (legajo 9389). Panel cerrado.

```

Figura 3: Panel interactivo: opción 1, opción 3, rechazo de la opción inválida 9 y salida por la opción 4.

3.4. Tarea 4: Verificador de sitios web

El script arma la lista de URL desde los argumentos o, si no hay ninguno, desde `sitios_9389.txt`, saltando líneas vacías y comentarios. Por cada URL pide a cURL únicamente el código de estado y lo clasifica por su primer dígito. Las secuencias de color solo se emiten si la salida es una terminal: redirigida a un archivo quedarían escritas como texto literal, y el log tiene que ser legible.

Las rutas del archivo de sitios y del log se resuelven contra la ubicación del script, de modo que el reporte siempre cae dentro del repositorio sin importar desde dónde se lo invoque.

3.4.1. Código de chequear_paginas.sh

```

#!/usr/bin/env bash
# chequear_paginas.sh - Tarea 4: verifica la disponibilidad HTTP de una lista de sitios.
# Uso      : ./chequear_paginas.sh [url ...]

```

```

# Sin argumentos, lee las URL de sitios_9389.txt
# Códigos: 0 si anduvo
# 1 si no hay argumentos y tampoco existe el archivo de sitios
# 4 si el archivo de sitios existe pero no tiene ninguna URL utilizable

LEGAJO="9389"

# Rutas contra la ubicación del script: el log cae siempre en el repositorio
DIR_SCRIPT="$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")" && pwd)"
RAIZ_REPO="$(dirname -- "${DIR_SCRIPT}")"
ARCHIVO_SITIOS="${RAIZ_REPO}/sitios_${LEGAJO}.txt"
ARCHIVO_LOG="${RAIZ_REPO}/logs/chequeo_${LEGAJO}.log"

TIEMPO_MAXIMO=10

# Los colores solo si la salida es terminal: redirigido, el ANSI quedaría como basura literal
if [ -t 1 ]; then
    VERDE='\033[0;32m'; AMARILLO='\033[0;33m'; ROJO='\033[0;31m'; NEUTRO='\033[0m'
else
    VERDE=""; AMARILLO=""; ROJO=""; NEUTRO=""
fi

urls=()

if [ "$#" -gt 0 ]; then
    urls=("$@")
    origen="argumentos de la línea de comandos"
else
    if [ ! -r "${ARCHIVO_SITIOS}" ]; then
        echo "ERROR: no se pasaron URL y no puedo leer '${ARCHIVO_SITIOS}'." >&2
        echo "Uso: $0 [url ...]" >&2
        exit 1
    fi
    while IFS= read -r linea; do
        linea="${linea#"${linea%%[![:space:]]*}"}" # recorta espacios al inicio
        linea="${linea%"${linea##*[![:space:]]}"}" # y al final
        [ -z "${linea}" ] && continue
        [ "${linea:0:1}" = "#" ] && continue
        urls+=("${linea}")
    done < "${ARCHIVO_SITIOS}"
    origen="${ARCHIVO_SITIOS}"
fi

if [ "${#urls[@]}" -eq 0 ]; then
    echo "ERROR: la lista de URL quedó vacía (origen: ${origen})." >&2
    exit 4
fi

if ! mkdir -p "$(dirname -- "${ARCHIVO_LOG}")"; then
    echo "ERROR: no pude crear el directorio del log." >&2
    exit 1
fi

{
    echo "== Chequeo del $(date '+%Y-%m-%d %H:%M:%S') == "
    echo "Origen de la lista: ${origen}"
} >> "${ARCHIVO_LOG}"

echo "Chequeando ${#urls[@]} sitio(s). Origen: ${origen}"
echo

for url in "${urls[@]"; do
    # Sin -L a propósito: siguiendo la redirección vería el 200 final y el 3xx no aparecería
    codigo="$(curl -s -o /dev/null -w '%{http_code}' --max-time "${TIEMPO_MAXIMO}" "${url}" 2>/dev/null)"

    # 000 no es un código HTTP sino la ausencia de respuesta (DNS, timeout): va en su rama
    # Etiquetas sin tildes: printf rellena contando bytes y en UTF-8 la vocal acentuada ocupa dos
    case "${codigo}" in
        2*) color="${VERDE}"; estado="OK" ;;
        3*) color="${AMARILLO}"; estado="REDIRIGIR" ;;
        4*) color="${ROJO}"; estado="ERROR CLIENTE" ;;
    esac

```

```

5*) color="${ROJO}";      estado="ERROR SERVIDOR";;
000) color="${ROJO}";      estado="SIN RESPUESTA" ;;
*) color="${ROJO}";        estado="DESCONOCIDO" ;;

esac

printf '%s%-3s%s %-14s %s\n' "${color}" "${codigo}" "${NEUTRO}" "${estado}" "${url}"
printf '%-3s %-14s %s\n' "${codigo}" "${estado}" "${url}" >> "${ARCHIVO_LOG}"

done

echo
echo "Reporte agregado a ${ARCHIVO_LOG}"
exit 0

```

3.4.2. Salida obtenida

```

.../tp2-cobis-9389 main ?+ 14:50
→ ./codigo/chequear_paginas.sh
Chequeando 4 sitio(s). Origen: /home/rolando/Clases/Admin_Sist_SL_2doAño_2doCua/AutomaScript/tp2-cobis-9389/sitios_9389.txt

200 OK          https://pedco.uncoma.edu.ar
301 REDIRIGE    http://www.uncoma.edu.ar
404 ERROR CLIENTE https://pedco.uncoma.edu.ar/pagina-inexistente-9389
000 SIN RESPUESTA https://sitio-que-no-existe-9389.uncoma.edu.ar

Reporte agregado a /home/rolando/Clases/Admin_Sist_SL_2doAño_2doCua/AutomaScript/tp2-cobis-9389/logs/chequeo_9389.log

```

Figura 4: Los cuatro estados: 200 en verde, 301 en amarillo, 404 y 000 en rojo.

3.4.3. Contenido de logs/chequeo_9389.log

```

== Chequeo del 2026-09-13 12:57:08 ==
Origen de la lista: /home/rolando/Clases/Admin_Sist_SL_2doAño_2doCua/AutomaScript/tp2-cobis-9389/sitios_9389.txt
200 OK          https://pedco.uncoma.edu.ar
301 REDIRIGE    http://www.uncoma.edu.ar
404 ERROR CLIENTE https://pedco.uncoma.edu.ar/pagina-inexistente-9389
000 SIN RESPUESTA https://sitio-que-no-existe-9389.uncoma.edu.ar
== Chequeo del 2026-09-13 14:50:07 ==
Origen de la lista: /home/rolando/Clases/Admin_Sist_SL_2doAño_2doCua/AutomaScript/tp2-cobis-9389/sitios_9389.txt
200 OK          https://pedco.uncoma.edu.ar
301 REDIRIGE    http://www.uncoma.edu.ar
404 ERROR CLIENTE https://pedco.uncoma.edu.ar/pagina-inexistente-9389
000 SIN RESPUESTA https://sitio-que-no-existe-9389.uncoma.edu.ar

```

3.5. Tarea 5: Respalador con prevención de ejecución simultánea

El bloqueo se toma con `mkdir`, que es la pieza central de la tarea: preguntar si el directorio existe y crearlo son una sola operación del núcleo, indivisible. Con un archivo común la comprobación y la escritura serían dos pasos separados, y entre uno y otro hay una ventana en la que dos procesos pueden superar la comprobación antes de que ninguno haya escrito nada; los dos se crearían dueños del bloqueo.

El `trap` se registra recién después de haber tomado el bloqueo. Registrado antes, una salida temprana del script (por ejemplo, la que ocurre al detectar que el bloqueo ya estaba tomado) borraría el directorio de bloqueo del proceso que realmente lo tiene.

El respaldo copia con `cp --parents` para conservar la ruta relativa de cada archivo: sin eso, dos archivos homónimos en carpetas distintas se pisarían dentro del temporal y el respaldo guardaría uno solo, en silencio. El empaquetado usa `tar -C` para que el tarball no recree el árbol `tmp/backup_9389/` al restaurarse.

3.5.1. Código de backup_manager.sh

```

#!/usr/bin/env bash
# backup_manager.sh - Tarea 5: respalda lo modificado en las últimas 24 h,
#                      impidiendo que dos instancias corran a la vez.

```

```

# Uso      : sudo ./backup_manager.sh
# Códigos: 0 si anduvo
#          7 si no puedo escribir en /var/lock (falta sudo)
#          9 si ya hay otra instancia en ejecución

LEGAJO="9389"
DIRECTORIO_BLOQUEO="/var/lock/backup_${LEGAJO}.lock"
DIRECTORIO_TEMPORAL="/tmp/backup_${LEGAJO}"

DIR_SCRIPT="$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")" && pwd)"
RAIZ_REPO="$(dirname -- "${DIR_SCRIPT}")"
DIRECTORIO_LOGS="${RAIZ_REPO}/logs"
FECHA="$(date '+%Y%m%d_%H%M%S')"
TARBALL="${DIRECTORIO_LOGS}/backup_${LEGAJO}_${FECHA}.tar.gz"

# mkdir como semáforo: preguntar si existe y crear son una sola operación del kernel
if ! mkdir "${DIRECTORIO_BLOQUEO}" 2>/dev/null; then
    # mkdir también falla por permisos: sin distinguir, informaría un bloqueo que no existe
    if [ -d "${DIRECTORIO_BLOQUEO}" ]; then
        echo "ERROR: ya hay una instancia en ejecución (bloqueo: ${DIRECTORIO_BLOQUEO})." >&2
        echo "Si está seguro de que no es así, elimine ese directorio a mano." >&2
        exit 9
    fi
    echo "ERROR: no pude crear el bloqueo en ${DIRECTORIO_BLOQUEO}." >&2
    echo "/var/lock pertenece a root: ejecute el script con sudo." >&2
    exit 7
fi

# El trap se registra recién acá: antes, una salida temprana borraría el bloqueo ajeno
trap 'rmdir "${DIRECTORIO_BLOQUEO}"' EXIT

echo "Bloqueo tomado: ${DIRECTORIO_BLOQUEO}"

if ! mkdir -p "${DIRECTORIO_TEMPORAL}" "${DIRECTORIO_LOGS}"; then
    echo "ERROR: no pude preparar los directorios de trabajo." >&2
    exit 1
fi

cd -- "${RAIZ_REPO}" || { echo "ERROR: no pude entrar a ${RAIZ_REPO}." >&2; exit 1; }

copiados=0
while IFS= read -r -d '' archivo; do
    # --parents conserva la ruta: sin eso, dos homónimos en carpetas distintas se pisan
    if cp -a --parents -- "${archivo}" "${DIRECTORIO_TEMPORAL}/" 2>/dev/null; then
        copiados=$(( copiados + 1 ))
    else
        echo "AVISO: no pude copiar '${archivo}'." >&2
    fi
done < <(find . -type f -mtime -1 -not -path './.git/*' -print0 2>/dev/null)

if [ "${copiados}" -eq 0 ]; then
    echo "No hay archivos modificados en las últimas 24 horas. No se genera tarball."
    exit 0
fi

echo "Archivos copiados a ${DIRECTORIO_TEMPORAL}: ${copiados}"

# -C para guardar rutas relativas al temporal y no recrear tmp/backup_9389/ al restaurar
if ! tar -czf "${TARBALL}" -C "${DIRECTORIO_TEMPORAL}" .; then
    echo "ERROR: falló el empaquetado en ${TARBALL}." >&2
    exit 1
fi

echo "Respaldo generado: ${TARBALL} $(du -h "${TARBALL}" | cut -f1)"

# Corriendo con sudo todo queda a nombre de root, incluido el tarball dentro del repositorio
if [ "${EUID}" -eq 0 ] && [ -n "${SUDO_USER:-}" ]; then
    chown -R -- "${SUDO_USER}" "${TARBALL}" "${DIRECTORIO_TEMPORAL}"
    echo "Propiedad de los artefactos devuelta a ${SUDO_USER}."
fi

exit 0

```

3.5.2. Salida obtenida

La primera ejecución toma el bloqueo, copia los archivos modificados en las últimas 24 horas y genera el tarball. A continuación se crea el directorio de bloqueo a mano, simulando una instancia en curso, y la segunda ejecución rebota con el mensaje de error y el código 9.

```

.../tp2-cobis-9389 main ? 14:55
→ sudo ./codigo/backup_manager.sh
Bloqueo tomado: /var/lock/backup_9389.lock
Archivos copiados a /tmp/backup_9389: 11
Respaldo generado: /home/rolando/Clases/Admin_Sist_SL_2doAño_2doCua/AutomaScript/tp2-cobis-9389/logs/backup_9389_2026
0913_145522.tar.gz (332K)
Propiedad de los artefactos devuelta a rolando.
.../tp2-cobis-9389 main ? 14:55
→ sudo mkdir /var/lock/backup_9389.lock
.../tp2-cobis-9389 main ? 14:55
→ sudo ./codigo/backup_manager.sh
ERROR: ya hay una instancia en ejecución (bloqueo: /var/lock/backup_9389.lock).
Si está seguro de que no es así, elimine ese directorio a mano.
.../tp2-cobis-9389 main ? 14:56
→ sudo rmdir /var/lock/backup_9389.lock

```

Figura 5: Respaldo exitoso y rechazo de la segunda instancia por bloqueo tomado.

4. Pruebas realizadas

Script	Caso probado	Resultado
pdf_version.sh	Directorio inexistente	Código 1
pdf_version.sh	Archivo sin cabecera %PDF	Aviso, sin abortar
pdf_version.sh	marco.pdf frente al filtro RC	Se analiza
organizador.sh	Sin argumentos	Código 1
organizador.sh	Directorio inexistente	Código 1
organizador.sh	paquete.tar.gz	A comprimidos/
organizador.sh	foto.JPG en mayúsculas	A imagenes/
organizador.sh	datos.csv sin categoría	A otros/
monitorear.sh	Opción fuera de rango (9)	Aviso, menú sigue
monitorear.sh	Fin de entrada (Ctrl-D)	Cierra con saludo
chequear_paginas.sh	URL que responde 200	Verde
chequear_paginas.sh	URL que redirige 301	Amarillo
chequear_paginas.sh	URL inexistente 404	Rojo
chequear_paginas.sh	Dominio que no resuelve	000, rojo
backup_manager.sh	Sin sudo	Código 7
backup_manager.sh	Bloqueo ya tomado	Código 9
backup_manager.sh	Ejecución normal	Tarball en logs/

Los cinco scripts pasan shellcheck 0.11.0 sin advertencias.